

Visvesvaraya Technological University

Belagavi – 590018, Karnataka



Project Report On

Design And Implement a Priority based token passing arbiter

Submitted in partial fulfilment of the requirements for the IV semester

Mini Project Work

Bachelor of Engineering

in

Electronics and Communication Engineering

of

Visvesvaraya Technological University, Belagavi.

By

Mohith M

1CD23EC192

PuvanRaj S

1CD24EC413

Rajesh V

1CD24EC414

Hemanth Kumar V

1CD24EC405

Under the Guidance of

Mr. Vijay Kumar

**SoC Design &
Verification Engineer**

External Guide

Elevium



Under the Guidance of

Prof. Indu

Assistant Professor

Internal Guide

Dept. Of ECE, CIT



Department of Electronic and Communication Engineering

CAMBRIDGE INSTITUTE OF TECHNOLOGY

KR Puram, Bengaluru 560036

2024-25

CAMBRIDGE INSTITUTE OF TECHNOLOGY

KP Puram, Bengaluru – 560036

Department of Electronic and Communication Engineering



CERTIFICATE

It is Certified that **Mr. Mohith M, Mr. Rajesh V, Mr. Hemanth Kumar and Mr. PuvanRaj.S** bearing USN: **1CD23EC192, 1CD24EC414, 1CD24EC405 1CD24EC413** respectively, are bonafide students of Cambridge Institute of Technology, and have completed requirements of the project entitled **“Design and Implement a priority-based token passed arbiter”** in partial fulfillment of the requirements for IV semester Bachelor of Engineering in Electronics and Communication Engineering of Visvesvaraya Technological University, Belagavi during academic year 2024-25. It is certified that all Corrections/Suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work Phase-2 prescribed for the Bachelor of Engineering degree.

Mr. Vijay Kumar
SoC Design &
Verification Engineer
External Guide
Elevium

Mrs. Indu
Assistant Professor
Internal Guide
Dept. of ECE. CITech

Dr. Shivapanchakshari T C
Head of the Dept.
Dept. of ECE. CITech

Dr.G.Indumathi
Principal

External Viva
Name of the Examiners

CITech
Signature with date

- 1.
- 2.

DECLARATION

We, **Mr.Rajesh V, Mr. PuvanRaj , Mr. Mohith M and Mr. Hemanth Kumar** bearing **USN: 1CD24EC414, 1CD24EC413, 1CD23EC192 and 1CD24EC405** respectively, are students of IV semester, Electronics and Communication Engineering, Cambridge Institute of Technology, hereby declare that the project entitled **““Design And Implement a priority based token passed arbiter”** has been carried out by us and submitted in partial fulfillment of the course requirements project work Phase-2 of IV semester Bachelor of **Engineering in Electronics and Communication Engineering** as prescribed by **Visvesvaraya Technological University, Belagavi**, during the academic year 2024-2025.

We also declare that, to the best of our knowledge and belief, the work reported here does not form part of any other report on the basis of which a degree or award was conferred on an earlier occasion on this by any other student.

Date:

Place:

Rajesh V (1CD24EC414)

PuvanRaj S (1CD24EC413)

Mohith M (1CD23EC192)

Hemanth Kumar V

(1CD24EC405)

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to Shri **D. K. MOHAN, Chairman**, Cambridge Group of Institutions, Bangalore, India for providing excellent Infrastructure and Academic Environment at CITECH without which this work would not have been possible.

I extremely thankful to Dr. G. **INDUMATHI, PRINCIPAL**, Cambridge Institute of Technology, Bengaluru for providing me Academic ambience and Laboratory facilities to in and everlasting motivation to carry out this work and shaping our careers.

I express my sincere gratitude to **Dr. SHIVAPANCHAKSHARI T G. HOD**, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for his stimulating guidance, continuous encouragement and motivation throughout the course of present work.

I also wish to extend my thanks to our Internal Guide **Prof. Indu**, Assistant Professor, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for their unstilted support, valuable guidance and help throughout the seminar related work.

I also wish to extend my thanks to **our External Guide Mr. Vijay kumar**, Elevium, Bengaluru for his critical, insightful comments, guidance and constructive suggestion and support to improve the quality of this seminar work.

I would like to thank all the faculty members and non-teaching staff of dept. of ECE and Elevium team for their constant support. And I would like to thank our parents and friends for their constant moral and financial support.

Rajesh V (1CD24EC414)

PuvanRaj (1CD24EC413)

Mohith M (1CD23EC192)

Hemanth Kumar V

(1CD24EC405)

Chapter 1

ABSTRACT

1. Objective Justification

"This project aims to simulate and validate the design behavior of a token-based priority arbiter..."

- Why? Arbitration is crucial in digital systems where multiple modules (like CPUs, memory controllers, DMA engines) compete for shared resources. Without efficient arbitration, conflicts can arise, leading to incorrect behavior or system inefficiencies.
- How? Simulating the arbiter in SystemVerilog allows for thorough validation before real hardware implementation, ensuring correct logic and timing.

2. Relevance of Arbitration in Digital Systems

"Arbitration is a critical concept... such as a communication bus or memory."

- Arbitration ensures that only one requester accesses a shared resource at any given time, preventing bus contention, data corruption, or system deadlock.
- Examples include:
 - CPUs accessing shared memory
 - Peripherals communicating via SPI/I²C buses
 - Multiple DMA channels needing RAM access

3. Token Passing Mechanism

"The token passing arbiter resolves these contentions based on a rotating priority..."

- A token-based arbiter ensures fairness: the token moves in a circular fashion, avoiding starvation of low-priority requesters, unlike fixed-priority arbiters.
- Each cycle, only the requester holding the token (if it has a request) is granted access. If it has no request, the token moves to the next

4. Design Approach and Language Choice

"This project presents a System Verilog testbench designed to simulate..."

- SystemVerilog was chosen because:
 - It supports modern constructs for verification (e.g., assertions, randomization).
 - Enables clear modeling of hardware behavior.
 - Simplifies creation of self-checking, reusable testbenches.

5. Arbiter Module Behavior

"...token arbiter, arbitrates access among four requesters (req[3:0])..."

- The token_arbiter module supports 4 requesters:
 - Inputs: req[3:0] representing the requests
 - Outputs: gnt[3:0] representing which requester is granted
- Behavior: At every clock edge, the token rotates. The current token position determines which requester can be granted if its request is active.

6. Testbench Initialization and Scenario Simulation

"The testbench initializes with a reset signal and applies a sequence of request patterns..."

- The testbench:
 - Initializes the system using a reset to ensure predictable startup behavior.
 - Applies varied request sequences to simulate different conflict scenarios.
 - This approach validates all paths and token behaviors over time.

7. Round-Robin Fairness

"The token is assumed to circulate in a fixed priority or round-robin manner..."

- Round-robin is implemented using a left-rotate on the token:
 - This ensures each requester gets a turn.
 - Prevents higher-index requests from being ignored, ensuring fair access.

8. Verification Goal

"...granting access to one requester at a time based on token position and request validity."

- Ensures:
 - Only one grant is active per cycle.
 - Correct grant matches token position.
 - Request signal is checked before granting access.

TABLE OF CONTENT

Chapter 1: Abstract

A brief overview of the project highlighting the scope, methodology, and significance.

Chapter 2: Introduction

Introduces the project domain, motivation, and background context.

Chapter 3: Problem Statement

Clearly defines the core technical or practical problem being addressed.

Chapter 4: Literature Survey

Presents a summary of existing research, papers, or systems related to the topic.

Chapter 5: Objectives of the Project

Lists the main goals and deliverables of the project.

Chapter 6: Block Diagram Description

Block Diagram (Text Representation) Explain the system architecture in text format representing data/control flow.

Chapter 7: Methodology

8.1 Requirement Analysis

List the hardware/software/practical requirements for implementation.

8.2 Design Specification

Describe functional specifications and design constraints.

8.3 RTL Implementation using Verilog

Outline the Register Transfer Level design coding in Verilog.

8.4 Testbench Development

Development of test environments to validate functionality

8.5 Simulation and Debugging

Process used to verify correctness and fix bugs.

8.6 Functional Verification

Confirms whether the design meets all requirements.

8.7 Documentation

Notes on systematic documentation of all stages.

Chapter 8: Development Tools Used

Include IDEs, simulators, synthesis tools (e.g., EDA Playground, etc.).

Chapter 9: Functional Analysis

12.1 Token Rotation Logic

Describe how the token passes among components.

12.2 Request-Grant Mapping

Explain mapping of request signals to grant outputs.

Chapter 10: Simulation Results

13.1 Annotated Waveform Explanation

Describe simulation waveform results.

13.2 Stimulated Output Waveform

Screenshots and explanation of final simulation outputs.

Chapter 11: Conclusion

Summarizes the overall implementation and performance of the project.

Chapter 12: Annexure

12.1 Annexure A: Verilog Design Code

12.2 Annexure B: Testbench Code

Chapter 2

Introduction

Priority-based token-passing arbitration is a hybrid arbitration mechanism used in digital systems to manage access to shared resources such as buses, memories, or communication channels. In such systems, multiple requesters may simultaneously seek access, and an arbiter is responsible for deciding who gets granted control. Token-passing arbitration operates by circulating a logical "token" among requesters; only the requester holding the token is allowed to access the shared resource. This ensures mutual exclusion and helps maintain fairness. When priorities are introduced into this mechanism, the arbiter not only considers token ownership but also the priority level of each requester. The token is granted to the highest-priority requester among those actively requesting access, making it suitable for time-sensitive or real-time applications. Unlike simple round-robin schemes, this method ensures that critical operations are not delayed by less important ones. Internally, the arbitration logic is often implemented using finite state machines, encoders, or shift registers that track the current token holder and evaluate requests based on priority. Priority-based token-passing arbitration strikes a balance between fairness and responsiveness, making it ideal for use in System-on-Chip (SoC) buses, DMA controllers, cache coherence networks, and on-chip communication fabrics where both performance and deterministic behavior are required.

Chapter 3

Problem Statement

In digital systems where multiple functional units or modules require access to a shared resource—such as a communication bus, memory unit, or data path—efficient arbitration is critical to ensure fair, conflict-free, and predictable access. Without proper arbitration, simultaneous access attempts can lead to data corruption, timing violations, or system stalls, which significantly degrade system performance and reliability.

Traditional fixed-priority arbiters are simple to implement but inherently unfair, as lower-priority requesters may be permanently starved if higher-priority requests persist. This is especially problematic in real-time or safety-critical applications, where every module must receive timely access. On the other hand, dynamic or fully programmable arbiters, while fairer, often require complex control logic, increased hardware area, and may introduce timing delays, making them less suitable for resource-constrained environments.

To address these challenges, this project proposes the design and implementation of a Priority-Based Token Passing Arbiter. The core idea is to use a circulating token among requesters, such that at any given clock cycle, only the requester holding the token is eligible to be granted access—but only if it has made a request. If the current token holder has no active request, the token moves to the next requester in a fixed circular order. This arbitration mechanism is:

- Fair – every requester eventually receives the token.
- Starvation-Free – lower-priority requesters are not indefinitely blocked.
- Deterministic – token movement is predictable and cyclic.
- Simple – control logic is minimal, reducing hardware overhead.

The goal is to simulate, verify, and validate this arbiter design using SystemVerilog, showcasing its functionality through a custom-built testbench. The design is scalable and can be easily adapted to systems with different numbers of requesters, making it suitable for SoC designs, network-on-chip architectures, and shared memory controllers.

By addressing the trade-off between fairness, complexity, and performance, this project aims to demonstrate how token-based arbitration can serve as an efficient and practical solution in modern digital systems requiring reliable resource sharing.

Chapter 4

Literature survey

Arbiters are fundamental components in digital systems for managing access to shared resources among multiple requesters. Various arbitration schemes have been developed, each offering trade-offs in terms of complexity, fairness, and hardware cost. This literature survey reviews the key concepts, methods, and previous works relevant to token-based arbitration mechanisms and their implementation using hardware description languages.

1.Fixed-Priority Arbitration

Fixed-priority schemes assign static priority levels to each requester. While simple and easy to implement, these schemes are prone to **starvation**, as lower-priority requesters may never be granted access under heavy traffic from high-priority sources.

- *Reference:* Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.

2.Round-Robin Arbitration

Round-robin arbitration rotates the priority among requesters in a cyclic manner, ensuring that all requesters eventually receive service. It provides fairness but may introduce latency for high-priority traffic

3.Token-Passing Arbitration

In token-based arbitration, a logical "token" circulates among requesters. A requester can only gain access to the resource if it holds the token and has an active request. This scheme ensures:

- Mutual exclusion (only one requester gets access at a time),
- Fairness (equal opportunity over time),
- Simplicity in control logic. Token-passing is often used in distributed systems and network-on-chip (NoC) environments.
- *Reference:* Dally, W. J., & Towles, B. (2004). *Principles and Practices of Interconnection Networks*. Morgan Kaufmann.

4.Verilog-Based Arbiter Design

Verilog HDL is commonly used for describing and simulating hardware logic at the RTL (Register Transfer Level). Token-based arbiters can be efficiently modeled using shift registers or rotating bits to simulate token movement and conditional grant logic.

- *Reference:* Palnitkar, S. (2003). *Verilog HDL: A Guide to Digital Design and Synthesis*. Pearson Education.

5.Real-World Applications

Token-based arbitration is widely employed in:

- Network protocols (e.g., Token Ring),
- Multi-core processor interconnects,
- Shared memory controllers, and
- Embedded systems with time-critical shared resources.

applications benefit from the deterministic and starvation-free nature of token-based

Summary

The literature indicates that while fixed and dynamic priority schemes have their use cases, token-based arbitration offers a compelling balance between fairness, simplicity, and hardware efficiency. The implementation of such arbiters in Verilog allows for fast simulation and integration into larger digital designs

Chapter 5

Objectives of the Project

The primary objective of this project is to design, implement, and simulate a Priority- Based Token Passing Arbiter using Verilog HDL for effective arbitration among multiple requesters contending for a shared resource. The arbiter ensures orderly, fair, and conflict-free access by employing a rotating token mechanism that grants priority based on the current token position.

Specific Objectives:

1. To design a hardware module that handles multiple simultaneous requests using a rotating token approach.
2. To implement the arbiter logic using Verilog HDL following register-transfer level (RTL) design principles.
3. To simulate and verify the functionality of the arbiter under different request conditions using a Verilog testbench.
4. To ensure fairness and eliminate the risk of starvation by rotating priority among requesters.
5. To analyze the behavior of the arbiter using waveform visualization tools (e.g., GTKWave) for functional validation.
6. To establish the potential of token-based arbitration as an efficient alternative to traditional fixed or dynamic priority systems in digital hardware applications.

Expected Outcomes

By the end of this project, the following outcomes are expected:

1. A functionally verified Verilog module implementing a priority-based token passing arbiter.
2. A testbench that simulates different request sequences to verify arbiter behavior.
3. Waveform outputs (via GTKWave) showing the correct rotation of the token and corresponding grant signals.
4. A clear demonstration that:
 - The arbiter provides fair access.
 - No requester experiences starvation. Only one request is granted per clock cycle.

Chapter 6

Block Diagram Description

1.Inputs:

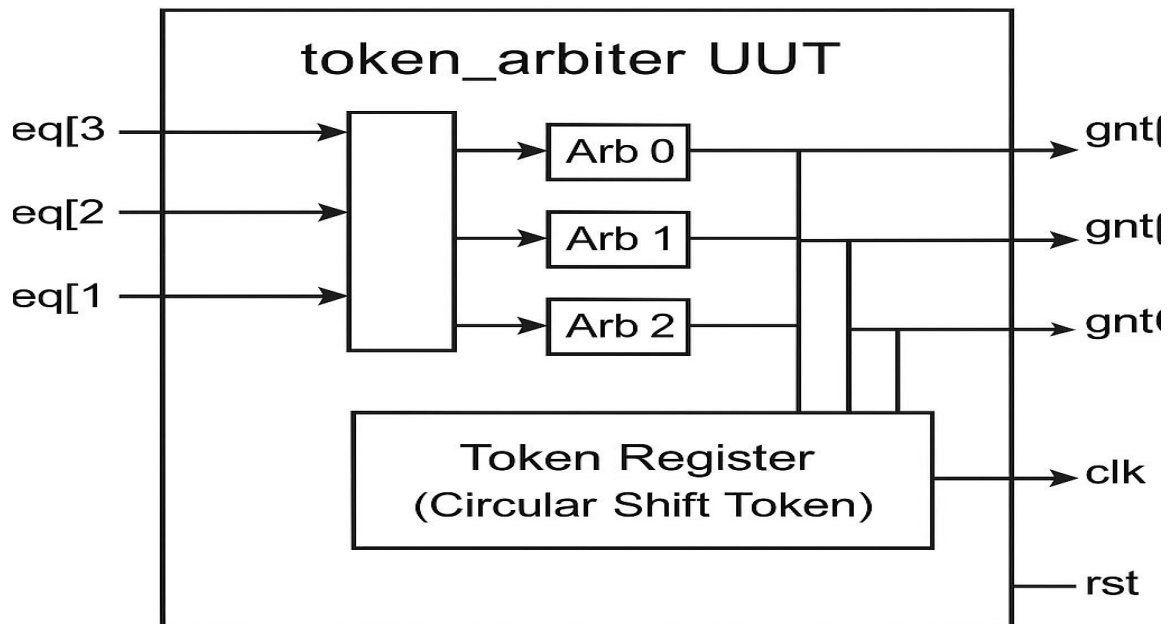
1. clk: Clock signal to drive token rotation.
2. rst: Active-high reset to initialize the token.
3. req[3:0]: 4-bit request signal from four different requesters.

2.Internal Logic:

- 1.Token Register: A 4-bit shift register that holds the current token. It rotates left on each clock cycle, passing control to the next requester in order.
- 2.Combinational Grant Logic: Checks if the current token matches an active request. If so, the corresponding grant bit is set

Output:

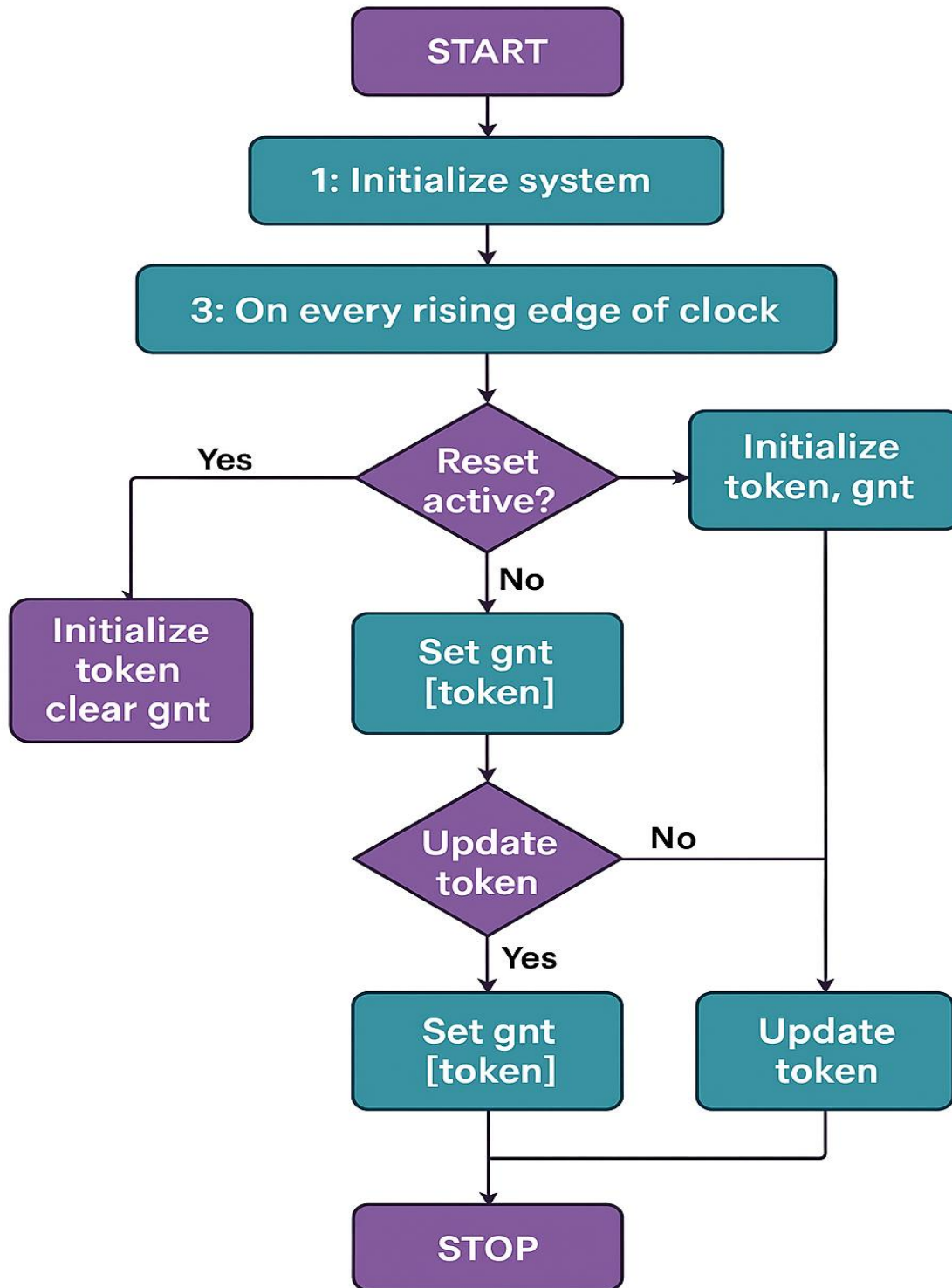
- 1.gnt [3:0]: 4-bit grant signal indicating which requester has been granted access.

Block**Diagram**

Step-by-Step Algorithm:

1. Start
2. Initialize system
 - On reset ($\text{rst} = 1$):
 - Set $\text{token} = 0$ (starting with requester 0)
 - Clear all grant lines: $\text{gnt} = 4'b0000$
3. On every rising edge of clock (clk)
 - If reset ($\text{rst} == 1$) is active:
 - Re-initialize $\text{token} = 0$
 - $\text{gnt} = 4'b0000$
 - Else:
 1. Clear previous grants: $\text{gnt} = 4'b0000$
 2. Token Matching Logic:
 - Check if the current token index (token) corresponds to an active request:
 - If $\text{req}[\text{token}] == 1$:
 - Set $\text{gnt}[\text{token}] = 1$
 - Else:
 - No grant is issued ($\text{gnt} = 0$)
 3. Update Token:
 - Move to the next token index in round-robin fashion:
 $\text{token} = (\text{token} + 1) \% 4$
4. Repeat Step 3 on every clock cycle
5. Stop (when simulation finishes or terminated)

Flow char



Chapter 7

Methodology

This project adopts a structured approach for designing, implementing, and validating a priority-based token passing arbiter using Verilog HDL. The overall process is divided into several key phases to ensure an efficient and verifiable design flow:

1.Requirement Identification

The initial phase involves understanding the need for arbitration in systems with multiple modules competing for a shared resource. The token- passing technique was selected for its simplicity and ability to ensure fair access. The functional requirements were defined as follows:

- Four independent request lines.
- Only one grant per clock cycle.
- Continuous token rotation to determine access.

2.Architectural Planning

An architectural framework was drafted to include:

- A rotating token mechanism to cyclically assign priority.
- Combinational logic to produce grant signals based on active requests and the current token position.
- A reset mechanism to ensure proper initialization of the token register.

3.RTL Design in Verilog HDL

The hardware logic was coded in Verilog at the Register Transfer Level (RTL). Key features include:

- A 4-bit token register implemented as a shift register to emulate token movement.
- Logic for decoding the token to generate corresponding grant outputs.
- A synchronous reset block to ensure deterministic start behavior

4. Testbench Development

A Verilog testbench was created to simulate various operational scenarios.

This involved:

- Clock and reset signal generation.
- Sequential application of different request patterns.
- Observing grant behavior under changing inputs.
- Ensuring synchronization between the clock and token logic.

5. Simulation and Debugging

The design was simulated using Icarus Verilog, and output signals were visualized via GTKWave. The process involved:

- Creating a VCD (Value Change Dump) file for waveform analysis.
- Verifying that only one grant is active per cycle.
- Confirming that the grant signal aligns with the requester currently holding the token and asserting a request.

6. Functional Validation

The arbiter's correctness was confirmed by evaluating the interactions between the token, request, and grant signals. Observations from the waveform confirmed:

- Cyclic token behavior.
- Starvation-free operation.
- Accurate grant issuance based on both token position and active request.

7. Project Documentation

All stages of development, from initial planning to verification, were documented comprehensively. This includes design rationale, test results, waveforms, and future improvement possibilities, forming the final project report

Chapter 8

Development Tools Used

Tool	Purpose
Verilog HDL	RTL hardware modeling
Icarus Verilog	Compilation and simulation
GTKWave	Waveform analysis
Text Editor (e.g., VSCode)	Code development

Tools and Technologies Used

Tool	Purpose
Verilog HDL	Hardware description and design
Icarus Verilog	Compilation and simulation
GTKWave	Waveform visualization
Linux / Windows	Development environment
Timescale	1ns / 1ps for simulation accuracy

Chapter 9

Functional Analysis

12.1. Token Rotation Logic

- The token is initialized to 4'b0001 after reset.
- It rotates left on every rising clock edge.

12.2. Request-Grant Mapping

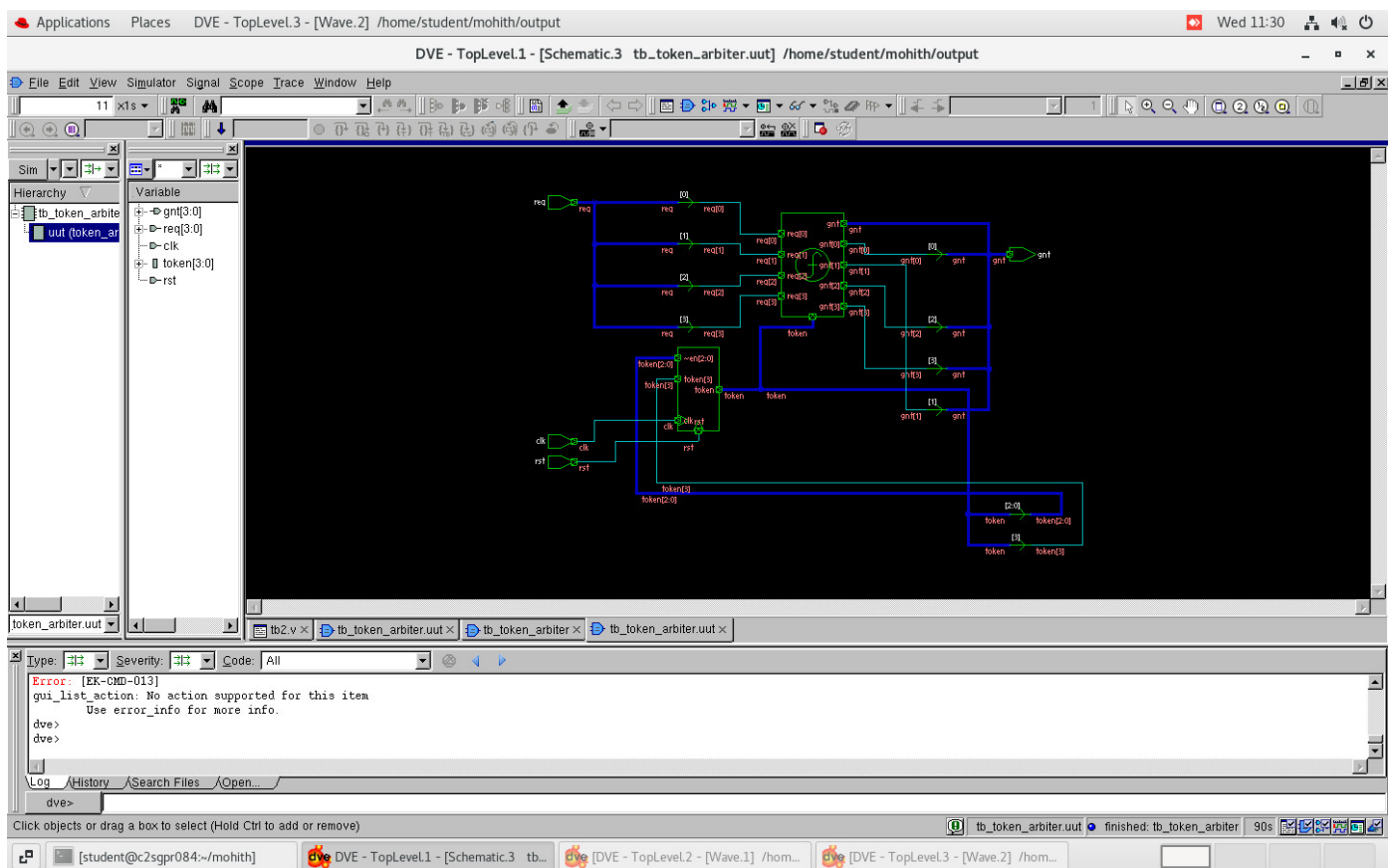
Time (ns)	Token	Request (req)	Grant (gnt)	Description
10	0001	0001	0001	Request 0 granted
50	0010	0010	0010	Request 1 granted
90	0100	0100	0100	Request 2 granted
130	1000	1000	1000	Request 3 granted
170	0001	0000	0000	No active requests

Chapter 10

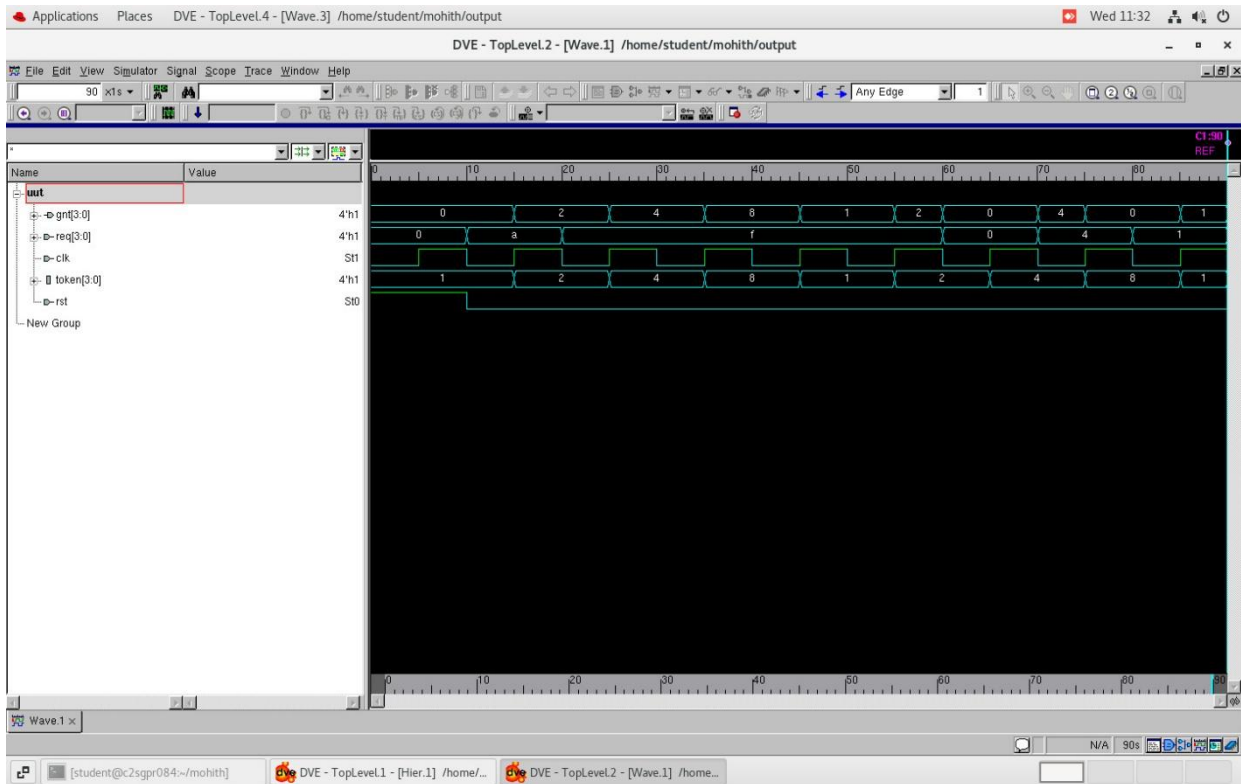
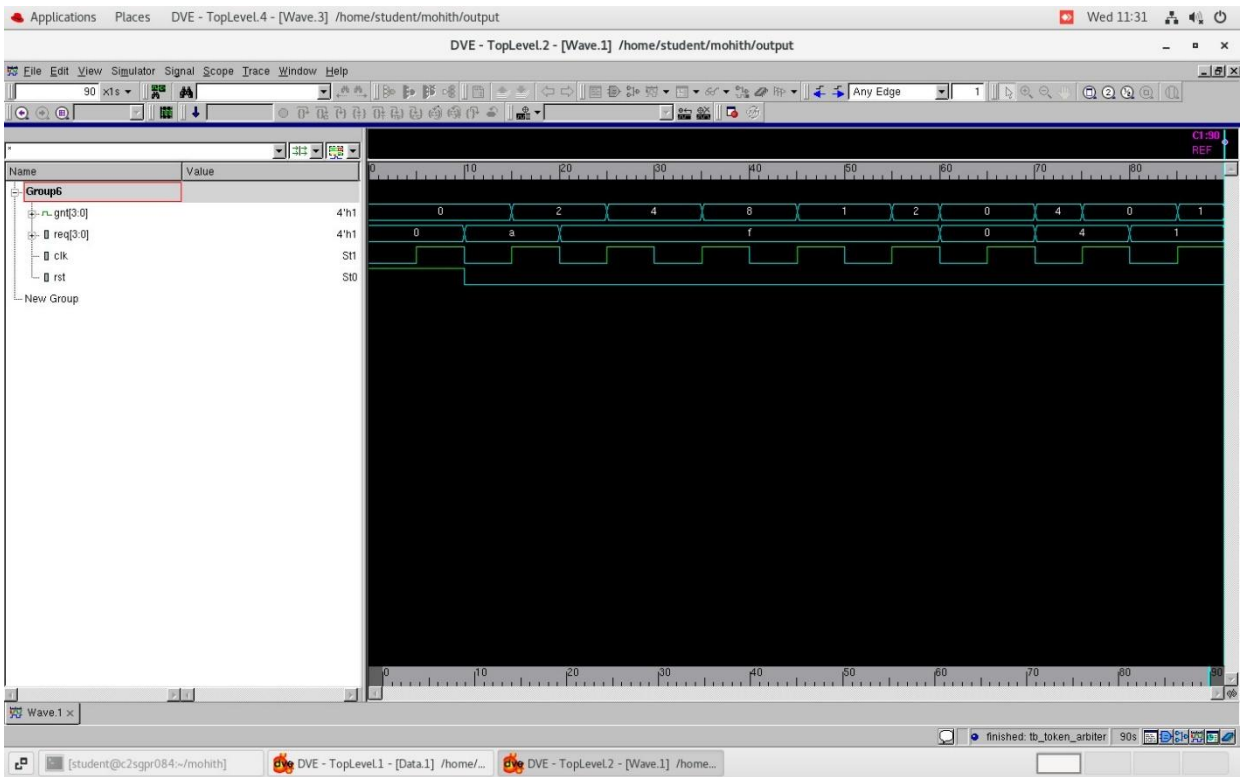
Simulation Results

- clk: Oscillates every 5ns, driving the token rotation.
- rst: Active initially to reset token to first position.
- req: Sequentially activates each requester every 40ns.
- gnt: Grants appear based on active requests and current token bit.

The waveform confirms correct arbitration behavior, with the grant signal following the token and request pattern precisely.

Stimulated

OUTPUT WAVEFORMS



Chapter 11

Conclusion

The priority-based token passing arbiter was successfully implemented and verified through simulation. It demonstrates fair and deterministic arbitration among multiple requesters. The design is scalable, conflict-free, and suitable for applications in SoCs, NoCs, and shared memory controllers.

Future Work

- 1.token size for more than four requesters.
- 2.Implement dynamic priority override or programmable priority modes.
- 3.Integrate timeout or deadlock detection mechanisms.
- 4.Synthesize the design on FPGA for hardware verification.

References

1. Morris Mano, *Digital Design*, Pearson Education
2. Verilog IEEE Standard Documentation
3. Icarus Verilog: <http://iverilog.icarus.com>
4. GTKWave: <http://gtkwave.sourceforge.net>
5. Online HDL resources and repositories

Chapter 12

Annexure

Annexure : Priority Arbiter Design Code

```
verilog
module priority_arbiter (
    input logic [3:0] req,
    output logic [3:0] grant
);
    always_comb begin
        grant = 4'b0000;
        if (req[0]) grant = 4'b0001;
        else if (req[1]) grant = 4'b0010;
        else if (req[2]) grant = 4'b0100;
        else if (req[3]) grant = 4'b1000;
    end
endmodule
```

Annexure : Priority Arbiter Testbench Code

```
module tb_token_arbiter;
    reg clk;
    reg rst;
    reg [3:0] req;
    wire [3:0] gnt;
    token_arbiter uut (
        .clk(clk),
        .rst(rst),
        .req(req),
        .gnt(gnt)
    );
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        rst = 1;
        req = 4'b0000;
        #10;
        rst = 0;
        req = 4'b1010; #10;
    end
endmodule
```

```
    req = 4'b1111; #40;
    req = 4'b0000; #10;
    req = 4'b0100; #10;
    req = 4'b0001; #10;
    $finish;
end
initial begin
    $dumpfile("token_arbiter.vcd");
    $dumpvars(0, tb_token_arbiter);
end
endmodule
```